A Reinforcement Learning Architecture for Spatio-Temporal Reallocation in Smart Container Terminals: Independent Proposal and Acceptance Policies with Counterfactual Cost Learning

Author Name[1]

Affiliation, Address email@example.com

Abstract. External truck jobs at a container terminal are assigned a yard block and an appointment time before the truck arrives, so congestion that develops afterwards cannot be answered by that fixed plan. This paper proposes a reinforcement learning architecture that revisits both the block and the arrival time of an already announced job during operation. The policy that requests a reallocation and the policy that accepts it on behalf of the destination block or time slot are modelled independently, and a central procedure commits only the candidates on which both agree under resource constraints. Because moving one job also changes the waiting of following trucks, the crane work sequence, and vessel supply, each policy is trained on the cost difference between discrete-event simulations that differ in a single action from the same state, whereas inference at operating time uses the trained networks alone. Evaluation uses a continuous synthetic environment in which external truck job demand is generated from a synthetic arrival curve informed by reported demand scales and by the strong time-of-day variation of truck arrivals. There the architecture lowered total operating cost by 14.7% against no reallocation, and remained lower than rule-based policies given the same action range (20 of 28 evaluation days, $p = 0.036$). Arrival-time adjustment accounted for 91.4% of that reduction, and the four most congested days accounted for 90.5% of it, indicating that spatio-temporal reallocation is most valuable as a selective response to observed congestion rather than as a continuous intervention.

Keywords: Smart port · Container terminal · Truck arrival management · Yard block reallocation · Counterfactual learning · Reinforcement learning

1 Introduction

Container volumes have kept growing while terminal land and equipment have not, and the resulting pressure on yard and gate operations has made the smart port a focus of investment and of research. That interest is not confined to automation hardware. Studies of port efficiency find that automation, intelligence, and environmental design act through different channels, so what a terminal

actually gains depends on how observed information is turned into operating decisions [1]. Attention has therefore stayed on the operational side of efficiency as much as on the equipment side, and work there continues along two lines. On the temporal side, truck appointment systems spread arrival peaks and let a terminal and carriers negotiate arrival times against their own costs [2, 3]. On the spatial side, yard operations research improves container placement, work sequencing, and the handling of crane interference [4–7]. Reinforcement learning has recently entered this layer as real-time crane dispatching [8].

These efforts share a boundary: they improve what happens *given* a working block and an appointment time that were fixed before the truck arrived. Yet during operation demand concentrates on particular slots or blocks and crane queues change with it, so an assignment that was reasonable when it was made may no longer be. This paper revisits the assignment itself, placing both *where* a job is placed and *when* the truck is asked to arrive inside one real-time decision on jobs that have already been announced. Such a reallocation does not change the cost of a single job alone: when a job moves, the waiting of following trucks, the crane work sequence, and vessel supply flow change as well, so a cost signal including the induced congestion is required, and difference rewards and multi-agent counterfactual baselines provide one by comparing an action with an alternative [9, 10]. Reallocation is also difficult to commit on the proposing side's benefit alone, since whether the destination can absorb the job must be judged separately; we therefore separate proposal and acceptance and leave consent and feasibility to a central procedure.

We ask whether spatio-temporal reallocation can be composed from independent proposal and acceptance policies, whether counterfactual learning can be separated from operational inference, whether the architecture reduces operating cost, and whether the roles of learning and of the action range can be told apart in the observed change. The contributions are an *independent policy structure*, a *spatio-temporal action range*, *counterfactual cost learning*, a *separation of learning and operation*, and a *decomposable evaluation* that reports execution, action range, and learning effect separately. Sect. 2 places these choices among existing work, Sect. 4 describes the environment, Sect. 3 the policies and the learning signal, and Sect. 5 the results.

2 Related Work

2.1 Smart Ports and Terminal Operations

Yen et al. measure port efficiency with data envelopment analysis and then explain the resulting scores through a Tobit regression on smart-port design attributes, separating *how efficient a port is* from *what accounts for the difference*; their treatment of automation, intelligence, and environmental design as distinct channels is what motivates studying the decision layer rather than the equipment layer [1].

On the temporal side, Chen et al. manage truck arrivals with time windows to relieve gate congestion, shaping the arrival profile instead of adding capacity [2].

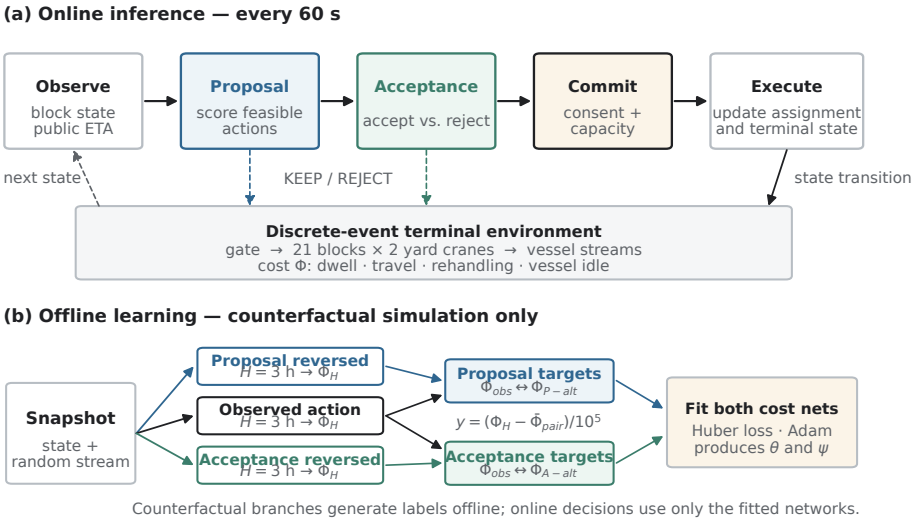

Fig. 1: Architecture and information flow. (a) Every 60 s the fitted proposal and acceptance networks score candidates, after which the central procedure commits only mutually accepted and feasible changes. Executing a decision advances the discrete-event terminal environment and produces the next observed state. (b) Offline learning clones the state and random stream immediately before a sampled decision, compares up to three worlds over a 3-h horizon, and fits the two networks to centred cost differences. The counterfactual branches are not invoked during online inference.

Phan and Kim keep that lever but drop the single-decision-maker assumption: trucking companies and the terminal each retain their own costs and coordinate scheduling and appointments between them, so the schedule emerges from two parties rather than one optimiser [3]. That decentralised setting is the direct antecedent of the acceptance policy used here, and the reason acceptance is modelled as a separate policy rather than folded into a single objective. Kourounioti and Polydoropoulou develop time-of-day models of truck arrivals from aggregate terminal data, establishing that arrivals are far from uniform across the day [11]; the value of shifting an arrival therefore depends on where in the arrival curve it starts and lands, and the environment of Sect. 4 reproduces that shape rather than a flat rate.

On the spatial side, Ng and Mak study yard crane scheduling in container terminals, sequencing the jobs a crane must serve so that truck waiting is reduced [4]. Speer and Fischer extend the problem to different automated yard crane systems, where cranes working the same block interfere with one another and the schedule must resolve that interference [5]. Petering and Murty take a terminal-wide view instead of a crane-wide one, using long-run simulation of a transshipment terminal to relate block length and crane deployment to overall performance [6]; their design—continuous operation over weeks with state car-

ried forward—is the one adopted below. Schwientek et al. compare dispatching methods on seaport container terminals in simulation and find their ranking to depend on terminal conditions [7], which is why Sect. 5 reports results under one fixed dispatching rule and examines its sensitivity separately.

2.2 Reinforcement Learning as Optimisation over Future Cost

A dispatching rule scores an action by what that action costs now. The difficulty in the problem addressed here is that the expense created by moving a job appears later and elsewhere—in the waiting of trucks that follow, in the crane work sequence, and in vessel supply—so a rule that prices only the immediate move cannot see it. Reinforcement learning is the natural tool precisely because it replaces the immediate cost with the cost an action leads to over a horizon, which is what makes a present decision answerable to its future consequences. Wang et al. apply a PPO-based approach to real-time yard crane scheduling across multiple blocks [8], placing that kind of optimisation inside the execution layer; the present work addresses the layer above it, where the block and the arrival time are decided.

Two properties of the networks used here follow from that choice. First, they do not output a distribution over actions. Each maps an observed state and one candidate action to the operating cost expected to follow from it, and the decision is the candidate of lowest predicted cost; this is the value-approximation route, in the sense in which deep Q-networks approximate action values with a neural network [12], rather than the policy-gradient route. Second, the approximator is a small multilayer perceptron. Multilayer feedforward networks are general function approximators [13], but that property alone does not select an architecture; the choice here follows from the data. The inputs are fixed-length numeric tables rather than images, text, or graphs; candidates are scored one at a time and independently of each other; the input carries no temporal ordering that a recurrent or attention model could exploit; the number of labelled decisions is small relative to the number of blocks, so a larger model would overfit; and one set of weights is shared across all blocks. Under those conditions a small perceptron is the appropriate first choice, and a transformer or graph network would add capacity the available labels cannot support.

Carrying future cost into a single agent's decision is the harder half when several parties act at once, because the horizon cost is joint. Wolpert and Tumer construct payoff functions for members of a collective that stay aligned with the global objective while remaining sensitive to the individual's own action—the difference-reward idea [9]. Foerster et al. realise the same idea in deep multi-agent learning by having a centralised critic marginalise one agent's action to form a counterfactual baseline [10]. Both obtain the future term from a learned value estimate. This paper takes the same decomposition but obtains the future term by measurement rather than estimation: the baseline is produced by re-running the discrete-event simulator from the same state with one action changed, so the regression target is a monetary cost difference accumulated over a fixed horizon rather than a bootstrapped advantage. The cost of that choice is offline

computation, and its benefit is that the learning signal is denominated in the same currency as the objective.

Kim et al. provide a related use of learning for resource-level control under environmental uncertainty in building demand response, and combine measured environmental records with synthetic parameters for the variation across individual units while stating the synthetic component declaratively [14]; Sect. 4.1 follows that practice.

3 Proposed Architecture

3.1 Notation, Actions, and Policies

Let the decision interval be $\delta = 60\,\mathrm{s}$ and let the current assignment of job i be $z_i = (b_i, \tau_i)$, with b_i the block and τ_i the announced arrival time. A reallocation action is $a_i = (b'_i, \Delta_i) \in \mathcal{A}_i(t)$, giving $z_i(a_i) = (b'_i, \tau_i + \Delta_i)$: keeping the assignment is $a_i^0 = (b_i, 0)$, block reallocation is $b'_i \neq b_i$, arrival-time adjustment is $\Delta_i > 0$, and candidates changing both are allowed. Outbound jobs are excluded from block reallocation because the container to be retrieved has a fixed location. With n_i the announcement time, g_i the gate-in time, and r_i an indicator of prior review, the set newly reviewed at t is $\mathcal{E}_t = \{i \mid n_i \leq t < n_i + \delta,\ g_i > t,\ r_i = 0\}$, so a job is reviewed at exactly one time point before it arrives. The proposal policy selects the action of lowest expected cost from its own block state and the job and action features o_i^P:

$$\hat{a}_i = \arg\min_{a \in \mathcal{A}_i(t)} \hat{c}_\theta^P(o_i^P, a), \qquad p_i = \mathbf{1}[\hat{a}_i \neq a_i^0]. \tag{1}$$

If a proposal is made, the destination block or target time slot becomes the accepting party. The acceptance policy compares the expected cost of accepting and rejecting from the proposal message and its own state o_i^A:

$$q_i = \mathbf{1}\left[\hat{c}_\psi^A(o_i^A, \hat{a}_i, \mathrm{accept}) \leq \hat{c}_\psi^A(o_i^A, \hat{a}_i, \mathrm{reject})\right]. \tag{2}$$

If the remaining capacity of the target resource is zero, the request is rejected before the network is consulted, and the set of mutually agreed candidates is $\mathcal{J}_t = \{i \in \mathcal{E}_t \mid p_i q_i = 1\}$.

3.2 Central Commitment Constraints

The central procedure creates no new candidate and commits only within $\mathcal{J}_t$, sorting agreed candidates by the predicted cost of the acceptance policy and breaking ties lexicographically. With $x_i \in \{0, 1\}$ the commitment indicator, $F_b(t)$ the free slots of block b, and $K_k(t)$ the remaining quota of slot k:

$$x_i \leq p_i q_i, \quad \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[b'_i = b] \leq F_b(t), \quad \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[k(i) = k] \leq K_k(t), \quad \sum_{t \in \mathcal{T}} \mathbf{1}[i \in \mathcal{E}_t] \leq 1. \tag{3}$$

Block capacity is enforced here; where time-slot quota data are unavailable we set $K_k(t) = \infty$ and read that condition as the range of what time adjustment could provide. All committed assignments are applied at the same instant, so a job committed earlier does not change the state seen by later candidates. Two conditions therefore hold by construction: only candidates carrying both a proposal and an acceptance are committed, and each job is reviewed exactly once.

3.3 Policy Models and Inputs

The two functions $\hat{c}^P_\theta$ and $\hat{c}^A_\psi$ of Sect. 3.1 are neural networks, and their role is worth stating precisely: neither produces a probability over actions. Each maps an observed state and one candidate to the cost expected to follow from committing that candidate, so the policy is an arg min over a learned cost rather than a sampled action, and the same network is evaluated once per candidate. Both are multilayer perceptrons that score fixed-length features per candidate, with one set of weights shared across blocks: 21-dimensional input for the proposal policy and 16 for the acceptance policy, two hidden layers of 64 units with ReLU, and 5,633 and 5,313 parameters respectively—fewer than 11,000 in total. Inputs comprise current waiting, inventory, and remaining crane work, the public expected arrival time, the route difference, and the announced volume around the target time. Unrealised actual arrival and completion times, and the simultaneous responses of other accepting parties, are excluded.

Fig. 2 shows the shape of one such network. A candidate enters as a fixed-length vector $x \in \mathbb{R}^{d_{in}}$, passes through three linear layers of which the first two are followed by a non-linearity, and leaves as a single number: the cost expected to follow from that candidate. Two consequences matter for the architecture around it. Because the output is one scalar per candidate rather than a distribution over an action set, the same network serves a proposal whose candidate list changes from epoch to epoch—blocks with free slots, deferral steps still ahead of the truck—without retraining for each list length. And because the width is the same for every block, one set of weights is shared across the yard: a block does not learn its own model, it learns how a block of its current state should read a candidate. The two policies differ only in d_{in}, which is 21 for proposal and 16 for acceptance.

3.4 Counterfactual Cost Labels

For each sampled decision we clone the state and random stream immediately before branching and run up to three worlds for three hours:

$$\phi^F_i = \Phi_H(\text{observed}), \quad \phi^P_i = \Phi_H(\text{proposal reversed}), \quad \phi^A_i = \Phi_H(\text{acceptance reversed}), \tag{4}$$

with $H = 10{,}800\,\text{s}$. Decisions without an acceptance request produce only the proposal comparison. For each policy the two actions are centred on their

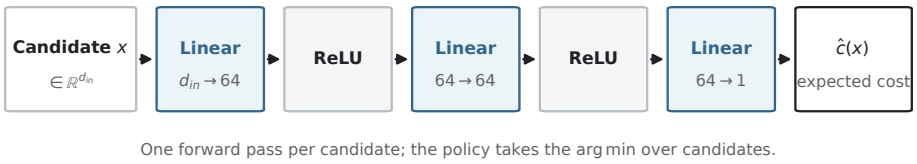

Fig. 2: Structure of one cost network. The input is a fixed-length feature vector for a single candidate and the output is the cost expected to follow from that candidate, so a decision evaluates the same network once per candidate and takes the arg min. The proposal and acceptance policies share this shape and differ only in the input dimension, 21 and 16 respectively.

mean $\bar{\phi}_i^P = (\phi_i^F + \phi_i^P)/2$ and divided by KRW 100,000, giving targets $y_{i,F}^P = (\phi_i^F - \bar{\phi}_i^P)/10^5$ and $y_{i,P}^P = (\phi_i^P - \bar{\phi}_i^P)/10^5$; the same centring is applied to (ϕ_i^F, ϕ_i^A) for the acceptance policy. Subtracting the mean removes the shared cost without changing the order of the two actions.

3.5 Training Settings and Operational Separation

The per-day sample cap is 64 decisions and the exploration probability decreases linearly from 0.50 to 0.05, with an 80/20 train–validation split at the decision level. Optimisation uses Adam (3×10^{-4}) and a Huber loss ($\beta = 1.0$), the latter because a few counterfactual differences are far larger than the rest and a squared error would let those samples dominate the fit [17]; minibatches of up to 256 rows, $\max(50, \min(600, 4 \times \text{rows}))$ updates, and gradients clipped at 1.0. Counterfactual branches run in parallel across CPU cores and each day's labels are replaced after that day's update. At evaluation the weights are frozen, exploration is disabled, and no counterfactual branch is invoked.

Training the pair took 4.7 hours of CPU time over 24 passes. At operating time a decision costs one forward pass per candidate through a network of the size given above, which is negligible beside the constraint check it accompanies; the expense of the method lies entirely in generating the counterfactual labels, and that work is offline and never enters the operating path.

4 Experimental Environment

4.1 A Literature-Calibrated Synthetic Environment

The architecture is evaluated in a synthetic environment rather than on the operating record of a particular terminal, with scales taken from the studies reviewed in Sect. 2: 20 blocks [7], several cranes per block [5], multi-week continuous operation [6], strong time-of-day variation in truck arrivals [11], three

vessel classes of differing call size [7], and a triangular range for crane handling time [6]. Within those ranges this study fixes 21 blocks, two cranes per block, 30 continuous days with state carried across the day boundary, three vessel classes, and 180 s of crane service time; the arrival curve is set out in Sect. 4.3. These fixed values are design choices of this study, and the citations establish only that they lie within the modelling range of existing work, not that they come from a specific port.

4.2 Terminal and Time Settings

The yard has 21 blocks with two cranes each (42 in total), a block capacity of 1,440 slots at 65% initial occupancy, and a one-dimensional layout with the gate and the quay on opposite sides. The decision interval is 60 s, operation runs for 30 continuous days, and the middle 28 days are compared. The crane work sequence gives priority to service work and, within it, to the shortest expected processing time. A day is a cut at which cost is separated, not a point at which the terminal stops: inventory, queues, crane state, and unfinished jobs carry over, with the first and last day connecting the boundaries.

4.3 External Truck Demand and the Arrival Process

External truck job demand is built from ranges reported in the literature rather than copied from it: prior work establishes that terminals operate at these demand magnitudes and that truck arrivals vary strongly over the day [11], whereas the particular curve used here is a synthetic construction of this study. The daily demand level is drawn from a discrete distribution over 3,500, 5,000, 7,500, 12,500, and 15,000 trucks with probabilities 0.30, 0.30, 0.25, 0.10, and 0.05 (Fig. 3a), the first three representing smooth, normal, and high demand and the last two congested and heavily congested conditions. Within a day the announcement time of each job is drawn from an arrival density $f(t)$ that combines a uniform base carrying 38% of the volume with three Gaussian components carrying the remainder (Fig. 3b):

$$f(t) = 0.38\,U(0,24) + 0.62 \sum_{m=1}^{3} w_m \mathcal{N}(t; \mu_m, \sigma_m^2), \tag{5}$$

with (μ_m, σ_m, w_m) equal to $(10, 1.5, .317)$, $(15, 2.5, .633)$, and $(21, 1, .05)$ and time measured in hours. The uniform base keeps arrivals non-zero at night, so the yard state carried across the day boundary is not an artefact of an empty night. Peak positions, widths, and weights are design values chosen so that congestion sensitivity can be compared across demand levels, not estimates fitted to a particular terminal. **The same curve shape is used for training and for evaluation**; only the demand level and the random draw differ between them, so the policy has not been tested against an arrival profile of a different shape.

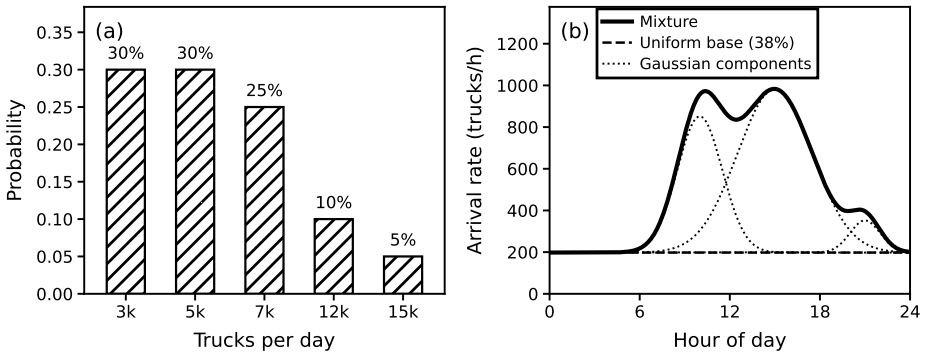

Fig. 3: External truck job demand. (a) The five daily demand levels and the probability with which each is drawn, independently for each day. (b) The time-of-day arrival process at 12,500 trucks per day, with the uniform base and the three Gaussian components drawn separately beneath the mixture they sum to. The literature supports the demand magnitudes and the fact that arrivals are far from uniform over the day, whereas the shape of the mixture is a construction of this study.

4.4 Vessels and Quay Operations

Vessels were divided into three classes: small (50,000 GT, 3,000 TEU, two quay cranes, KRW 149,500 idle cost per hour), medium (100,000 GT, 7,500 TEU, four, KRW 299,000), and large (150,000 GT, 14,000 TEU, six, KRW 448,500). The call volume of a vessel of capacity C TEU was sampled as $C \cdot u$ with $u \sim U[0.15, 0.30]$, and the daily composition was two small vessels and one medium vessel, to which one large vessel was added with probability 0.30. The throughput of one quay crane was set to 27.5 move/h, within the 24, 30, and 36 move/h range used in the literature [7]. Blocking across quay cranes was summed and divided by the number assigned, giving the idle time per vessel. On the yard side the reference service time of one crane is 180 s per move, so the nominal capacity is $\mu_{YC} = 21 \times 2 \times 3600/180 = 840$ moves per hour. A move is a container movement, not a truck: one truck job may require several moves and rehandles, and the realised service time varies with position and job type, so we report move units and truck units separately.

4.5 Cost Function

The total cost over the interval $[d, d+1)$ is the sum of four terms:

$$\Phi_d = C_{wait,d} + C_{move,d} + C_{rehandle,d} + C_{vessel,d}. \tag{6}$$

Truck dwell costs KRW 40,000 per hour with the same rate applied again beyond one hour, anchored on the safe trucking freight rate [15]; vessel idling costs KRW 2.99 per GT-hour, converted from the excess berthing rate of KRW 29.9 per 10 GT-hour [16]; additional crane movement (KRW 60,000 per hour)

and rehandling (KRW 2,000 each) are design values not taken from the literature. With h_i the dwell time of truck i, m the additional crane working time, R the number of rehandles, and T_v the policy-related idle time of vessel v:

$$C_{wait} = \sum_i 40{,}000\{h_i + \max(0, h_i - 1)\}, \quad C_{move} = 60{,}000\, m, \tag{7}$$

$$C_{rehandle} = 2{,}000\, R, \quad C_{vessel} = \sum_v 2.99\, G T_v T_v. \tag{8}$$

Structural vessel time that the reallocation policy cannot change, such as berthing and quarantine, is separated as a diagnostic value. All four terms are computed as differences of cumulative values at the same day boundary.

Two properties of the resulting environment were measured over the 30-day training run. First, waiting dominates: C_{wait} accounts for 96.4% of total cost, against 2.2% for rehandling, 1.2% for vessel idling, and 0.1% for additional crane movement. Any conclusion drawn from Φ is therefore governed almost entirely by the safe-freight-rate coefficient taken from regulation [15], and is insensitive to the two coefficients that are design values rather than literature values. Second, both the share of waiting and the service level worsen sharply with demand: waiting rises from 83.2% of cost at 3,500 trucks per day to 91.7% at 7,500 and 99.1% at 15,000, crowding out every other term. Across the month the mean turn time was 0.89 h and the 90th percentile 2.01 h, with 5.0% of 206,904 truck jobs unfinished on the day they were announced; but that 90th percentile ranges from 0.79 h on the lightest days to 7.49 h on the heaviest, a 9.5-fold spread. Congested days are thus not merely more expensive—they are expensive for a different reason, and this is the property the reallocation policy is asked to exploit.

5 Numerical Experiments

5.1 Scope of the Results

The results below come from the middle 28 days of a 30-day continuous diagnostic run generated with random seed 9,900,950, which does not overlap with training. Weights were frozen and the exploration probability set to zero, and all policies on a given day shared the same truck, vessel, and service random streams. Each day stores the four cost terms, the demand level, the transaction types, and the unfinished jobs; the source code, the raw per-policy results, and the figure scripts are released with the paper.

5.2 Confirmation of Architectural Operation

Four structural conditions—separate models for proposal and acceptance, no commitment on one-sided consent, no realised future arrival or completion time among policy inputs, and no state reset at the day boundary—follow from the implementation, the commitment constraints, and automated tests. Two more were

Table 1: Every policy on the same 28 days, sorted by cost reduction. The two rightmost columns compare each policy with the proposed policy day by day. Reading down the *action range* column shows the pattern: every policy that uses arrival time reduces cost, and every policy restricted to block reallocation sits near zero.

			vs. proposed	
Policy	Action range	Reduction	Days lower	p
Proposed policy	block + time	**+14.72%**	—	—
Proposed, time only	time	+13.44%	—	—
Untrained model	block + time	+7.97%	18/28	0.185
Rule: least-loaded slot	time	+3.41%	—	—
Rule: least-loaded block + slot	block + time	+1.51%	20/28	**0.036**
Rule: least slack	block	+0.53%	20/28	**0.036**
Rule: first-come-first-served	block	+0.17%	22/28	**0.004**
Rule: shortest processing time	block	+0.05%	21/28	**0.013**
Rule: net gain	block	−0.48%	21/28	**0.013**
Proposed, block only	block	−1.31%	—	—
No reallocation	—	0.00%	26/28	**<0.001**

confirmed in the runs themselves: counterfactual simulation calls during frozen-policy evaluation numbered **zero**, and the identity check, in which a branch that changes no action must reproduce the action and cost of the main run, passed 4/4 where counterfactual labels exist.

5.3 Paired Daily Comparison Across Policies

Every policy is run on the same random streams, following Kleijnen's analysis of common random numbers, so that a difference between two policies is not read off simulation noise [18]. The cost difference is the proposed policy minus the comparison policy; a negative value means the proposed policy costs less. Days are treated as independent comparison units and a two-sided sign test is used (Table 1). A Wilcoxon signed-rank test on the same pairs, which uses the size of each daily difference and not only its sign, reaches the same conclusion throughout and is never weaker ($p \leq 0.017$ against every rule-based policy).

Over 28 days the total cost was KRW 10.00 bn without reallocation and KRW 8.52 bn with the proposed policy, a reduction of KRW 1.471 bn (14.7%). The four rule-based policies above use block reallocation only, so this comparison reflects the potential of the architecture as a whole; a comparison matched on action range is treated in Sect. 5.5. The days on which the proposed policy costs more are few and small, whereas the days of largest reduction are those of congested demand.

Table 2: Where the reduction comes from, by demand level (KRW bn, against no reallocation). Columns three to five allow one action at a time; the last column is the share of the total that training adds, the trained model measured against the untrained one. Arrival-time adjustment produces almost all of the reduction and does so on the four congested days, block reallocation helps under light demand but turns negative when demand is highest, and the contribution of training follows the same congested-day pattern.

Daily demand	Days	Both actions	Time only	Block only	Due to training
3,500	12	0.047	−0.013	0.057	0.056
5,000	8	0.060	0.029	0.067	0.029
7,500	4	0.032	0.008	0.000	−0.049
12,500	2	0.611	0.573	−0.014	0.144
15,000	2	0.722	0.747	−0.241	0.495
Total	**28**	**1.471**	**1.344**	**−0.131**	**0.675**
share of total		100%	**91.4%**	—	**45.9%**

5.4 Demand Level and the Decomposition of Actions

Table 2 decomposes the reduction by demand level. Of the total, 90.5% occurred on the four days with demand of 12,500 and 15,000 trucks, which supports developing the architecture towards a policy that *intervenes selectively when congestion is observed* rather than continuously. Allowing only one action at a time over the same 28 days separates the two contributions: arrival-time adjustment accounts for 91.4% of the total reduction and almost all of it on those four days, whereas the block-only condition reduces cost under smooth and normal demand (KRW 0.124 bn) but raises it at the highest demand, leaving a net increase of KRW 0.131 bn. The policy can therefore be optimised towards adjusting the relative use of spatial and temporal actions with the demand level. Time-slot quotas were open in this diagnostic run, so the arrival-time figures describe the *range* of what time shifting could provide; a sensitivity analysis with finite quotas would refine the scope of application.

5.5 Rule-Based Policies of the Same Action Range

Sect. 5.4 raises the question of what a rule achieves when given the same action. Two rules were added by transferring the *least-loaded* criterion from the spatial to the temporal axis, introducing no new parameter and sharing the congestion condition of the other rules; Table 1 places them among the rest. Three observations follow. Arrival-time adjustment lowers cost significantly even under a rule (+3.41%, $p = 0.0125$ against no reallocation), so the action itself carries value. When the action range is matched the proposed policy still costs less than the rule (20/28, $p = 0.036$). Yet on the arrival-time axis alone the *rule was lower on more days* (the proposed policy 8/28) while the total was about four times

lower for the proposed policy, because the rule is marginally lower under smooth demand whereas the proposed policy is 3.8–5.6 times lower on the congested days: the learned selection was optimised *not towards winning frequently but towards reducing cost sharply on the expensive days*, and both indicators must be reported for this to be visible. The rule using both actions also weakens under congested demand, reproducing the property of Sect. 5.4 and suggesting it belongs to the action rather than to a particular policy.

5.6 Learning Effect and Dispatching Sensitivity

Before the cost comparison, the fit itself. Over the 30 training iterations the proposal network's Huber loss fell from 0.079 to 0.029 on training decisions and from 0.268 to 0.119 on held-out ones, and the acceptance network's from 0.171 to 0.027 and from 0.315 to 0.136, taking means over the first and last five iterations with targets in units of KRW 100,000. Validation loss therefore falls with training, so the fit generalises to decisions the networks did not see; it nonetheless ends about four times the training loss, which is what roughly 56 labelled decisions per iteration permits and is a further reason for keeping the model small. The share of decisions whose counterfactual difference was exactly zero fell from 51.4% to 37.6% as exploration decayed, and 4,671 counterfactual worlds were run in total.

From KRW 10.00 bn without reallocation, the untrained model reached KRW 9.20 bn and the trained model KRW 8.52 bn, so training adds KRW 0.675 bn, 45.9% of the total. Counting days, however, the trained model was lower on only 18 of 28 ($p = 0.1849$), and the mean daily difference (KRW -24.1 m) is eighteen times the median (-1.3 m). The two statistics disagree because the effect is concentrated rather than spread over the month: the four congested days carry 94.6% of it and the trained model was lower on all four, against 9/12, 5/8, and 0/4 on the lighter days, whereas the 24 remaining days give a mean of -1.5 m with a bootstrap interval that spans zero. Resampling the daily differences within demand strata, which respects the fixed demand mix of the stage, gives a 95% interval of $[-32, -17]$ m per day for the mean difference; resampling without strata widens it to $[-52, -3]$ m, with 99.2% of resamples remaining negative. Both exclude zero, though the narrow one is narrow partly by construction, since each congested stratum holds only two days. The conclusion is therefore bounded: learning contributes a reduction that is large in total and without exception on congested days, and counting days understates it because the policy was optimised to cut cost sharply when it is expensive rather than to win frequently—as observed for the rules in Sect. 5.5. Establishing it at a conventional significance level requires more congested days, not more days.

Separately, the ranking of crane dispatching rules reversed with demand—the shortest-processing-time rule in current use was 22–25% more expensive than others under smooth and normal demand but the cheapest from 7,500 trucks upward, and lowest overall when weighted by the designed demand mix—so the effect of the reallocation policy should also be re-examined under other dispatching rules.

6 Conclusion

This paper developed a reinforcement learning architecture that revisits both the yard block and the arrival time of an already announced truck job during operation, extending distributed appointment negotiation [3] and yard crane operations [4, 5] into one spatio-temporal assignment problem. Because the cost of a reallocation appears later and elsewhere, each policy learns from a counterfactual cost difference measured over a fixed horizon rather than from an immediate move cost, which is how multi-agent counterfactual learning [9, 10] enters an operational decision; keeping that branch computation offline leaves the operating path as a small cost model and a constraint check.

Observations in the literature-calibrated environment indicate that the structure carries spatio-temporal reallocation through to execution, and that arrival-time adjustment and learned selection produce a direction of cost reduction under congested conditions, which supports the view that real-time information at a smart port can lead to assignment actions rather than monitoring alone [1]. The same direction held when the action range was matched, although on the arrival-time axis alone the rule was lower on more days and the proposed policy led only in total.

6.1 Limitations

Seven limitations bound these results. The environment is synthetic: its scale and variation are calibrated to reported ranges, but no operating record of a terminal was fitted, and the demand mix, the arrival-curve coefficients, and two of the four cost coefficients are design values. Training and evaluation moreover share the same arrival-curve shape, differing only in the demand level and the random draw, so the policy has not been shown to transfer to an arrival profile it never saw; varying peak times and widths during training and evaluating on a held-out curve is the natural next step. Congestion is rare on the stage—four of the 28 days—and that is exactly where the effect lives, so the statistics rest on a small number of expensive days; more congested days, rather than more days, is what would settle the learning contribution. The contribution of the consent structure itself has not been isolated: we confirmed that nothing is committed on one-sided consent, but we did not measure what removing the acceptance policy's veto would cost, so separating proposal from acceptance remains a design choice supported by its operation rather than by an ablation. Time-slot quotas were left open, so the arrival-time results describe an upper range rather than an executable schedule. The three-hour counterfactual horizon is derived from the design—a 30-minute review window, up to 120 minutes of deferral, and a 30-minute arrival-pressure margin—rather than taken from a reported value. Finally, the main comparison uses a single crane dispatching rule, and because the ranking of those rules reverses with demand, the size of the reallocation effect may shift under another one.

Each of these can be addressed one axis at a time: reflecting actual appointment quotas would make the executable range of time adjustment concrete,

and repeating a wider variety of congested conditions would delineate the contribution relative to the untrained model. Observation, proposal, acceptance, commitment, execution, and counterfactual learning have been organised into a single reproducible structure, which as automated equipment and appointment information spread can serve as a basis for connecting real-time congestion information to explainable resource assignment decisions.

References

1. Yen, B.T.H., et al.: How smart port design influences port efficiency: a DEA-Tobit approach. Res. Transp. Bus. Manag. **46**, 100862 (2023)
2. Chen, G., Govindan, K., Yang, Z.: Managing truck arrivals with time windows to alleviate gate congestion at container terminals. Int. J. Prod. Econ. **141**(1), 179–188 (2013)
3. Phan, M.-H., Kim, K.H.: Collaborative truck scheduling and appointments for trucking companies and container terminals. Transp. Res. B **86**, 37–50 (2016)
4. Ng, W.C., Mak, K.L.: Yard crane scheduling in port container terminals. Appl. Math. Model. **29**(3), 263–276 (2005)
5. Speer, U., Fischer, K.: Scheduling of different automated yard crane systems at container terminals. Transp. Sci. **51**(1), 305–324 (2017)
6. Petering, M.E.H., Murty, K.G.: Effect of block length and yard crane deployment systems on overall performance at a seaport container transshipment terminal. Comput. Oper. Res. **36**(5), 1711–1725 (2009)
7. Schwientek, A., Lange, A.-K., Jahn, C.: Simulation-based analysis of dispatching methods on seaport container terminals. ARGESIM Report **59**, 357–364 (2020)
8. Wang, W., et al.: Yard crane real-time scheduling among multi-block at terminal: a reinforcement learning based PPO approach. Transp. Res. E **207**, 104635 (2026)
9. Wolpert, D.H., Tumer, K.: Optimal payoff functions for members of collectives. Adv. Complex Syst. **4**(2–3), 265–280 (2001)
10. Foerster, J., et al.: Counterfactual multi-agent policy gradients. In: AAAI, vol. 32 (2018)
11. Kourounioti, I., Polydoropoulou, A.: Application of aggregate container terminal data for the development of time-of-day models predicting truck arrivals. EJTIR **18**(1) (2018)
12. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
13. Hornik, K.: Approximation capabilities of multilayer feedforward networks. Neural Netw. **4**(2), 251–257 (1991)
14. Kim, S., Mowakeaa, R., Kim, S.-J., Lim, H.: Building energy management for demand response using kernel lifelong learning. IEEE Access **8**, 82131–82141 (2020)
15. Ministry of Land, Infrastructure and Transport: Safe trucking freight rates for 2026, Notice 2026-402 (2026). (in Korean)
16. Ministry of Oceans and Fisheries: Port facility charges: berthing rates and calculation basis. (in Korean)
17. Huber, P.J.: Robust estimation of a location parameter. Ann. Math. Statist. **35**(1), 73–101 (1964)
18. Kleijnen, J.P.C.: Analyzing simulation experiments with common random numbers. Manag. Sci. **34**(1), 65–74 (1988)